kit 국립금오공과대학교
Kumoh National Institute of Technology

이산신호시스템설계 보고서

Mechanical-ball suspension in CT/DT

국립금오공과대학교 | 전자시스템전공

2조: 20학번 나승엽, 오세범 21학번 김태준(팀장), 이찬중

지도교수 : 양연모 | 2026.06.17

목차

kit 국립금오공과대학교
Kumoh National Institute of Technology

1. 목적
2. 시스템 요약
3. 역할 분담
4. 일정
5. CT Simulation
6. DT Simulation
7. DT Realization
8. 트러블슈팅
9. 한계 및 후속연구
10. 결론

1. 목적

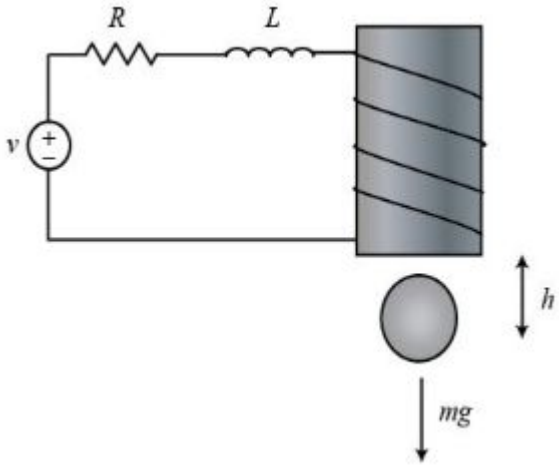
자기 부상 기술의 중요성

도심형 자기부상열차, 정밀 자기 베어링 등 마찰과 소음을 최소화하는 자기 부상 제어 기술 수요 증가.
이러한 시스템에서 불안정한 비선형 특성을 극복하기 위한 정밀 제어 알고리즘 설계 필수.



과제 수행 목표

Magnetic Ball 시스템 Free Body Diagram 모델링, 선형화.
CT/DT 영역에서의 상태 피드백 제어기 및 관측기 설계를 통해 실무적 제어 공학 역량 함양.



2. 시스템 요약

kit 국립금오공과대학교
Kumoh National Institute of Technology

개발 환경

[CT/DT Simulation]
Matlab
(Control System Toolbox
추가 설치)

[DT Realization]
Raspberry Pi 4,
Hardware Module,
VScode (SSH 원격 접속)

CT Simulation

기계 전자 시스템 모델링 및 동작점
(평형점)에서의 선형화.
상태 제어기 및 관측기 설계.

성능 비교:

개루프 시스템 vs 폐루프 시스템,
관측기 성능에 따른 변화,
초기 상태 오차에 따른 변화,
Ackermann vs LQG

DT Simulation

CT -> DT 로 시스템 이산화(ZOH).
추후 보드에 포팅을 위해 내장함수
배제 및 직접 구현 (c2d -> expm,
acker -> ackermann 공식 대입,
ss, lsim -> for문).

성능 비교:

샘플링 주기에 따른 변화,
관측기 초기 추정값에 따른 변화

DT Realization

HIL 시뮬레이션 시스템 구축.
유저 모드 -> 커널 모드 isr로
실시간성 향상.
Q-15 고정소수점 연산 최적화
대신 나노초 정수 스케일링으로
계산 지연 방지.

성능 비교:

샘플링 주기에 따른 변화,
유저 모드 vs 커널 모드

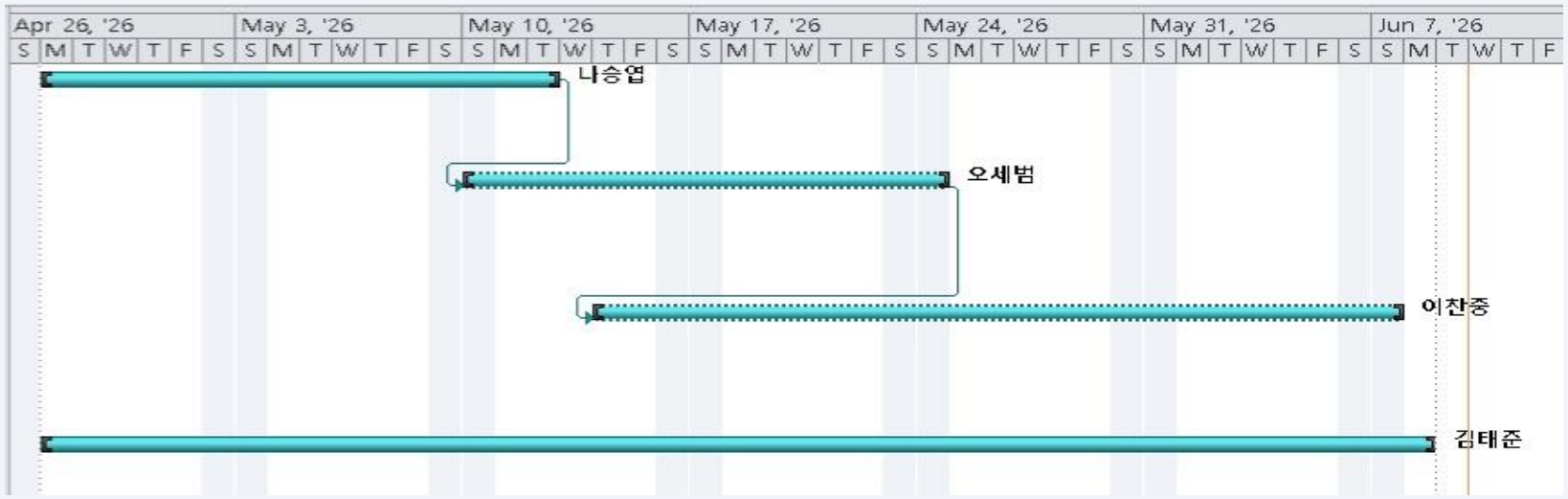
3. 역할 분담

kit 국립금오공과대학교 Kumoh National Institute of Technology

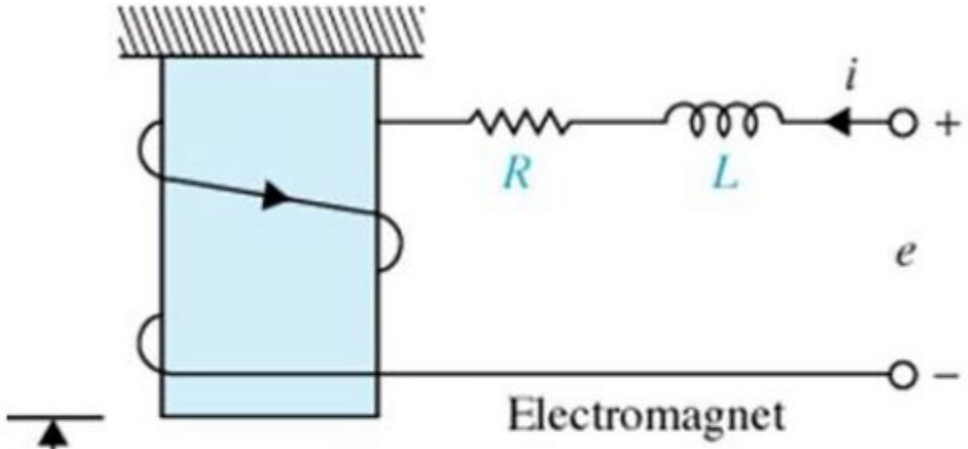
성명	주요 역할 및 수행 내용
나승엽	기계·전기 복합 시스템 수학적 모델링, 동작점 및 평형점 해석 수행
오세범	상태 피드백 제어기 설계, 안정도 판별 및 정상상태 오차 제거용 피드포워드 보상기 설계
이찬중	관측기 이득(L) 극점 배치 최적화, DT 변환 및 임베디드 포팅 후 커널 모드 ISR 구축
김태준(팀장)	프로젝트 총괄 (CT/DT Simulation 설계 및 분석, DT Realization 구현)

4. 일정

Task Name	Duration	Start	Finish	Predecessors	Resource Names
[CT sim] 기계·전기 복합 시스템 수학적 모델링, 동작점 및 평형점 해석 수행	12 days	Mon 4/27/26	Tue 5/12/26		나승엽
[CT sim] 상태 피드백 제어기 설계, 시스템 안정도 판별 후 및 정상 상태 오차 제거용 피드포워드 보상기 설계	12 days	Sun 5/10/26	Sun 5/24/26	1FS-2 days	오세범
[DT sim, realiztion] 관측기의 관측 게인(L) 극점 배치 최적화, 추후 커널 모드 ISR 구축, DT 변환 및 커널 포팅	18 days	Thu 5/14/26	Sun 6/7/26	2FS-3 days	이찬중
[All] 프로젝트 총괄	31 days	Mon 4/27/26	Mon 6/8/26		김태준



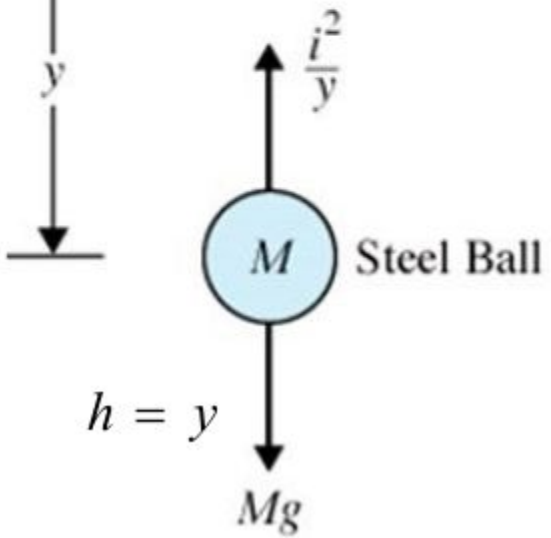
5. CT Simulation : 시스템 모델링



수학적 모델링

$$M\frac{d^2h}{dt^2} = Mg - K\frac{i^2}{h}$$

$$V = L\frac{di}{dt} + iR$$



e(t): input voltage
y(t): ball position (h)
i(t): winding current
R: winding resistance
L: winding inductance
M: mass of ball
g: gravitational acceleration (9.8 or 9.81m/sec)
K: magnetic force coefficient (K=1?)

상태 변수 선정

$$x_1(t) = y(t)$$
$$x_2(t) = dy(t)/dt = \dot{y}$$
$$x_3(t) = i(t)$$
$$u(t) = e(t)$$

5. CT Simulation : 선형화

$$\frac{dx_1(t)}{dt} = x_2(t)$$

$$\frac{dx_2(t)}{dt} = g - \frac{1}{M}\frac{x_3^2(t)}{x_1(t)}$$

$$\frac{dx_3(t)}{dt} = -\frac{R}{L}x_3(t) + \frac{1}{L}u(t)$$

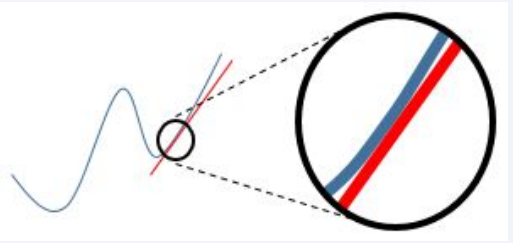
동작점(평형점) 기반 선형화

$$0 = x_{20}$$

$$0 = g - \frac{1}{M}\frac{x_{30}^2}{x_{10}} \rightarrow x_{30}^2 = Mgx_{10} \rightarrow x_{30} = \sqrt{Mgy_0}$$

$$0 = \frac{u_0 - Rx_{30}}{L} \rightarrow u_0 = Rx_{30} \rightarrow u_0 = R\sqrt{Mgy_0}$$

$$A = \left.\frac{\partial f}{\partial x}\right|_{\substack{x=x_0 \\ u=u_0}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} \\ \frac{\partial f_3}{\partial x_1} & \frac{\partial f_3}{\partial x_2} & \frac{\partial f_3}{\partial x_3} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ \frac{g}{x_{10}} & 0 & -2\sqrt{\frac{g}{Mx_{10}}} \\ 0 & 0 & -\frac{R}{L} \end{bmatrix}, \quad B = \left.\frac{\partial f}{\partial u}\right|_{\substack{x=x_0 \\ u=u_0}} = \begin{bmatrix} \frac{\partial f_1}{\partial u} \\ \frac{\partial f_2}{\partial u} \\ \frac{\partial f_3}{\partial u} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \frac{1}{L} \end{bmatrix}$$



자코비안 행렬을 사용하여 최종적으로 동작점 기반 미소영역에서 선형화됨

5. CT Simulation : 선형화

```
%% [Part 1] System Definition & Linearization
% 1. Physical Parameters (물리적 하드웨어 스펙)
M = 5.0;          % Mass of the ball (kg)
L = 0.01;         % Winding inductance (H)
R = 1.0;          % Winding resistance (Ohm)
g = 9.81;         % Gravitational acceleration (m/s^2) - 9.81로 정밀도 상향

% 2. Operating Point / Equilibrium Point (동작점/평형점 설정)
% 제어기가 가장 잘 동작하기를 원하는 기준 높이를 설정합니다.
y0 = 0.01;        % Operating point position:

% 평형 상태에서의 각 상태 변수 값 (수식 적용)
x10 = y0;         % 위치 (m)
x20 = 0;          % 속도 (m/s) - 평형상태이므로 무조건 0
x30 = sqrt(M * g * x10); % 평형 유지 전류 (A)

% 완벽한 평형점 상태 벡터
x0 = [x10; x20; x30]

% 3. State-Space Model Setup (Jacobian Linearization)
% 수작업 하드코딩이 아닌, 수식 기반의 동적 행렬 생성
A = [   0,              1,              0;
        g/x10,          0,              -2*sqrt(g / (M * x10));
        0,              0,              -R/L ]

B = [   0;
        0;
        1/L ]

C = [   1,              0,              0 ];

D = 0;
```

동작점 = 1cm 기반 선형화

```
x0 =

    0.0100
         0
    0.7004

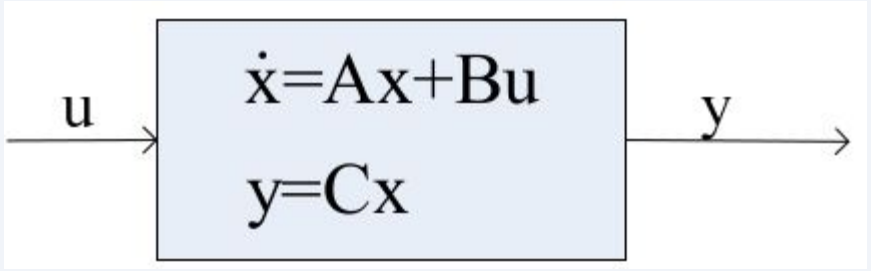

A =

         0    1.0000         0
  981.0000         0  -28.0143
         0         0 -100.0000


B =

      0
      0
    100
```

5. CT Simulation : 개루프 극점 확인

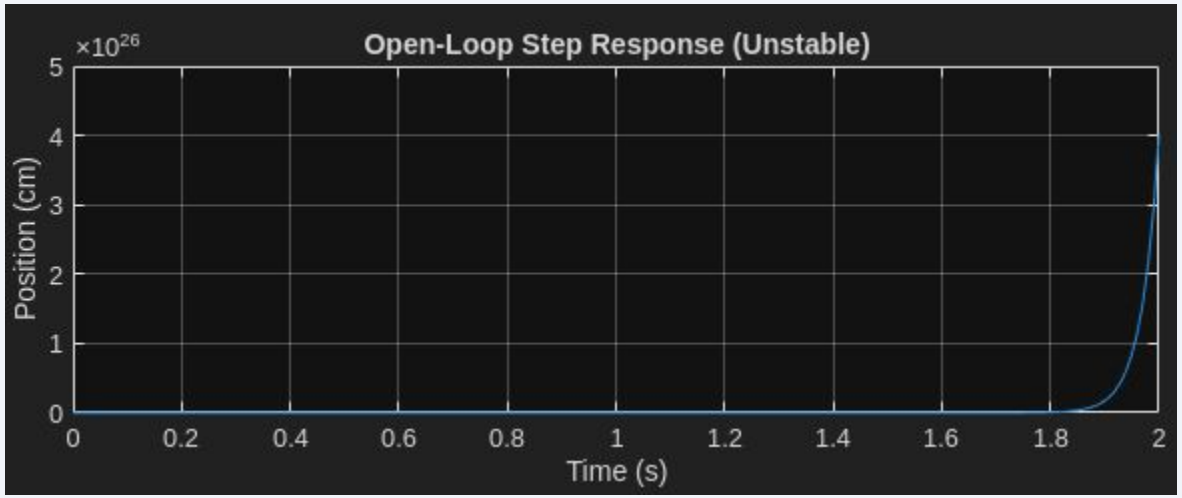



```
disp('--- Open-Loop Poles ---');
disp(eig(A));
```

시스템 행렬의 고유값으로 시스템 극점을 구한다

```
--- Open-Loop Poles ---
   31.3209
  -31.3209
 -100.0000
```

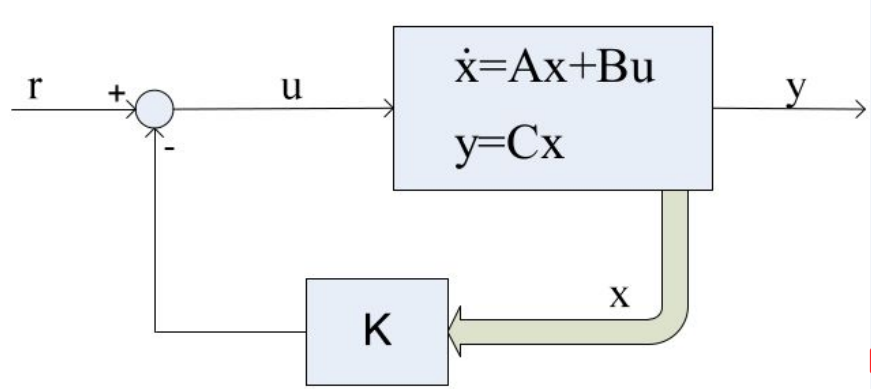
우반면에 하나 존재



시스템의 고유응답이 불안정하므로
어느 입력을 넣더라도 발산한다

-> 상태 제어기 설계 필요

5. CT Simulation : 제어기 설계



Ackermann's formula

$$K = e_n^T P_c^{-1} \alpha_c(A)$$

where $e_n^T = [0\ 0\ 0\ \ldots\ 0\ 1]$

매트랩 acker, place 함수로 대체 가능

```
%% [Part 2] Ackermann Controller Design (Pole Placement)
% 1. Performance Specifications
Mp_c = 0.2;              % 20% overshoot
ts_c = 0.5;              % Settling time

% 2. Target Pole Calculation
zeta_c = -log(Mp_c) / sqrt(pi^2 + (log(Mp_c))^2);
wn_c = 3 / (zeta_c * ts_c);

sigma_c = -zeta_c * wn_c;
wd_c = wn_c * sqrt(max(0, 1 - zeta_c^2));

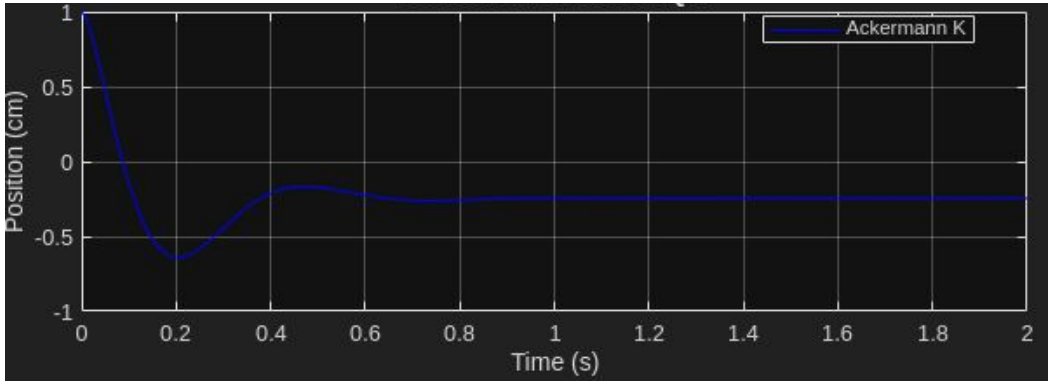
cp1 = sigma_c + 1i*wd_c; % Dominant poles
cp2 = sigma_c - 1i*wd_c;
cp3 = -5 * wn_c;         % Fast non-dominant pole

% 3. Calculate K and Nbar
K_acker = acker(A, B, [cp1 cp2 cp3]);
```

```
disp('--- Closed-Loop Poles (Controller) ---');
disp('Ackermann:'); disp(eig(A - B*K_acker));
```

```
--- Closed-Loop Poles (Controller) ---
Ackermann:
 -65.7967 + 0.0000i
  -6.0000 +11.7119i
  -6.0000 -11.7119i
```

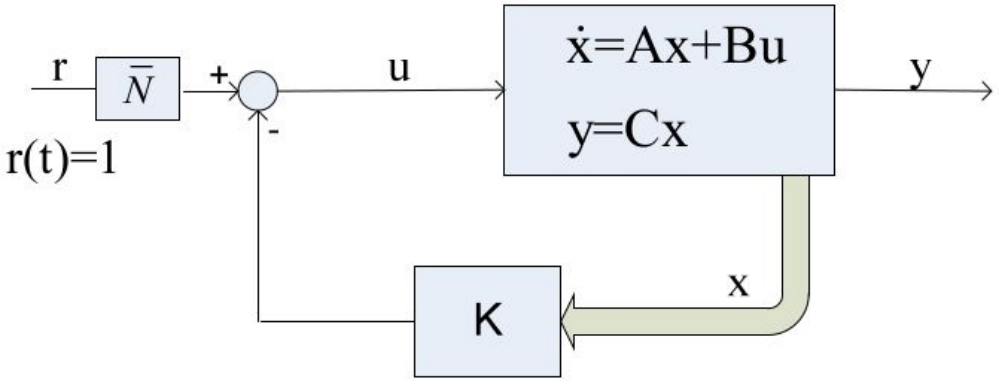
원하는 극점으로 배치



안정해졌으나 정상상태 오차 존재

-> 피드백 이득 K 뿐만 아니라 피드포워드 이득 Nbar도 필요

5. CT Simulation : 제어기 설계



$$0 = Ax_{ss} + Bu_{ss}$$

$$y_{ss} = Cx_{ss} + Du_{ss}$$

$$y_{ss} = r_{ss}$$

$$x_{ss} = N_x r_{ss} \text{ and } u_{ss} = N_u r_{ss}$$

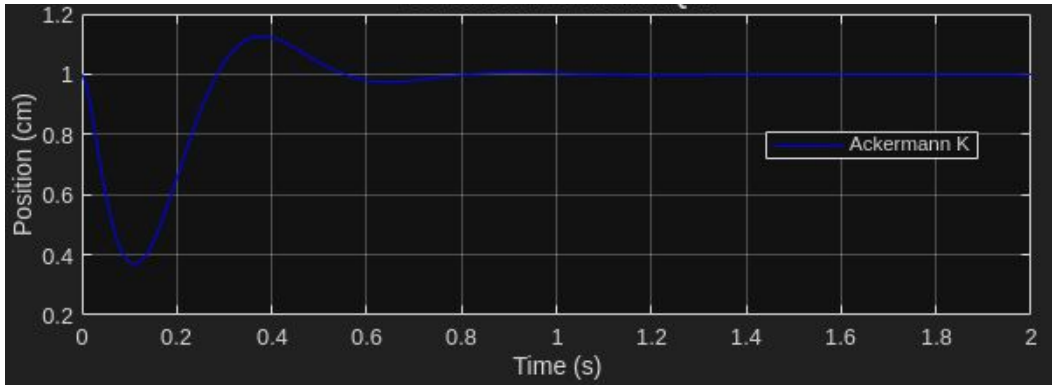
$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} N_x \\ N_u \end{bmatrix} r_{ss} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} r_{ss}$$

$$\begin{bmatrix} N_x \\ N_u \end{bmatrix} = \begin{bmatrix} A & B \\ C & D \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\bar{N} = N_u + KN_x \rightarrow u = -Kx + \bar{N}r$$

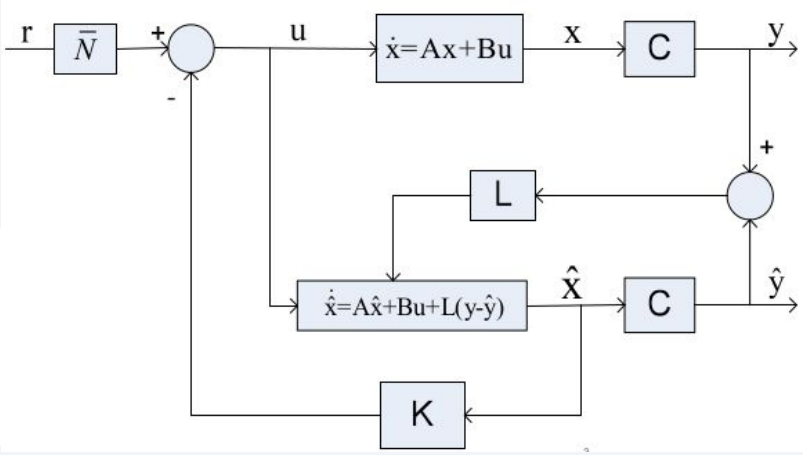
```
Z_vec = [0; 0; 0; 1];
N_mat = inv([A, B; C, 0]) * Z_vec;
Nx = N_mat(1:3);                    % 정상 상태에서 필요한 상태 (위치, 속도, 전류)
Nu = N_mat(4);                      % 정상 상태에서 필요한 기본 전압
Nbar_acker = K_acker * Nx + Nu;     % 최종 피드포워드 게인 산출
% Nbar_acker = -1 / (C * ((A - B*K_acker) \ B)); % Steady-state scaling

% 4. Closed-Loop Simulation (Without Observer)
sys_acker = ss(A - B*K_acker, B*Nbar_acker, C, 0);
[y_acker, ~, x_acker] = lsim(sys_acker, r, t, x0);
```



정상상태 오차가 제거되어 r = 1 과 동일

5. CT Simulation : 관측기 설계



실제 시스템에서는 측정할 수 없는 상태 변수가 존재
우리 프로젝트에서는 속도와 전류가 해당
-> 상태 관측기 설계 필요

관측기 스펙 정의
(제어기 극점보다 5배 이상이 안정적)

Ackermann's formula

$$L=\alpha_o(A)P_o^{-1}e_n$$

$$\text{where } e_n=[0\ 0\ 0\ \ldots\ 0\ 1]^T$$

```
%% [Part 3] Ackermann Observer Design (Luenberger)
% 1. Observer Specifications (Faster than controller)
Mp_o = 0.1;
ts_o = 0.1;

zeta_o = -log(Mp_o) / sqrt(pi^2 + (log(Mp_o))^2);
wn_o = 3 / (zeta_o * ts_o);

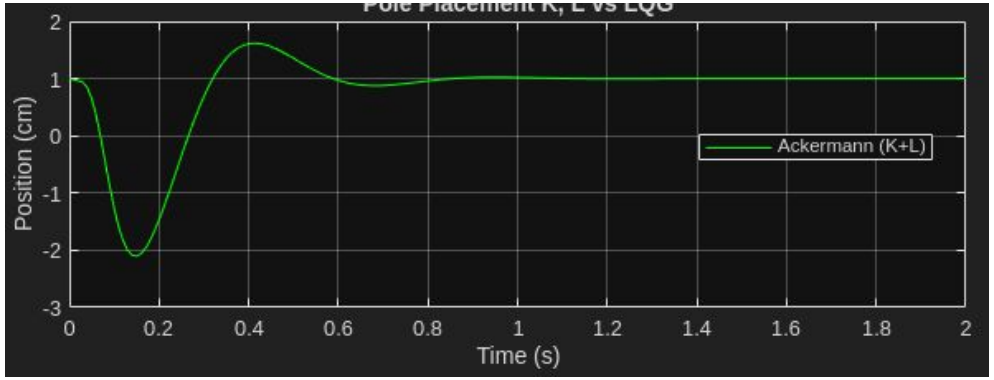
sigma_o = -zeta_o * wn_o;
wd_o = wn_o * sqrt(max(0, 1 - zeta_o^2));

op1 = sigma_o + 1i*wd_o;
op2 = sigma_o - 1i*wd_o;
op3 = -5 * wn_o;

% 2. Calculate Observer Gain L
L_acker = acker(A', C', [op1 op2 op3])';

% 3. Augmented System (Plant + Observer) Simulation
At_acker = [A - B*K_acker,          B*K_acker;
            zeros(size(A)),         A - L_acker*C];
Bt_acker = [B * Nbar_acker;
            zeros(size(B))];
Ct_acker = [C, zeros(size(C))];

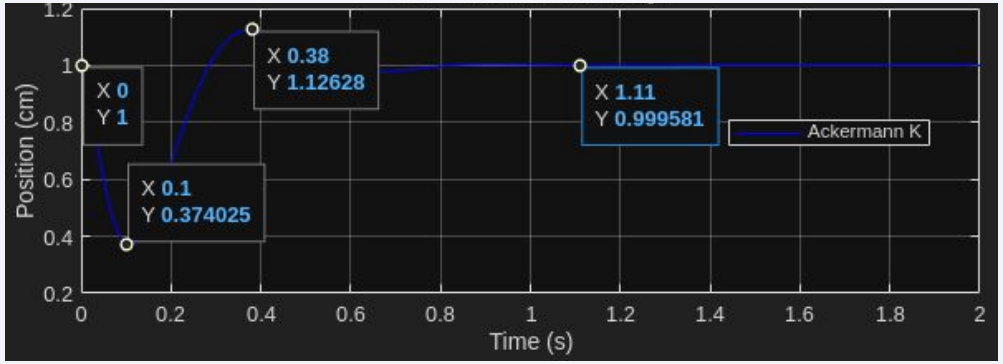
sys_obs_acker = ss(At_acker, Bt_acker, Ct_acker, 0);
[y_obs_acker, ~, x_obs_acker] = lsim(sys_obs_acker, r, t, [x0; e]);
```



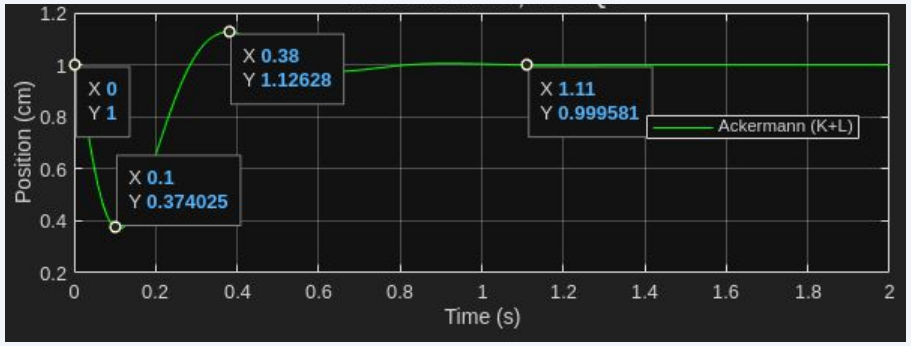
Separation Principle에 따라 관측기는 제어기의 안정성과 정상상태에 영향 x

하지만 관측기의 응답 속도나 초기 추정값에 따라,
시스템의 과도응답이 달라짐
(현재는 상태 변수의 초기 추정값을 모두 0으로 설정)

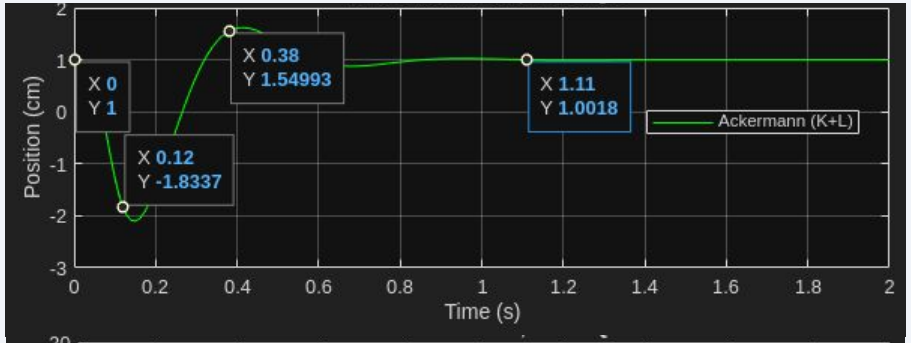
5. CT Simulation : 관측기 설계



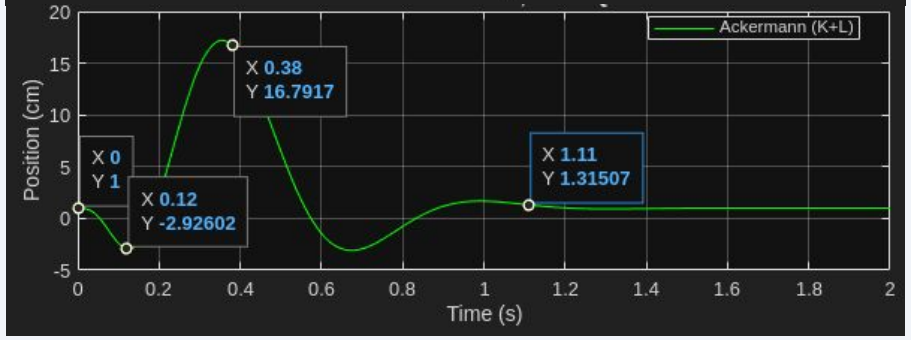
관측기의 초기 추정 값과 정착시간 등에 따라
전체 시스템의 성능이 달라진다




```
x0_obs = x0;
% x0_obs = [0; 0; 0];
e = x0 - x0_obs;
```




```
% x0_obs = x0;
x0_obs = [0; 0; 0];
e = x0 - x0_obs;
Mp_o = 0.1;
ts_o = 0.1;
```




```
% x0_obs = x0;
x0_obs = [0; 0; 0];
e = x0 - x0_obs;
Mp_o = 0.1;
ts_o = 0.5;
```

5. CT Simulation : 최적 제어

```
%% [Part 4] LQR Design (Optimal Control)
% 1. Weighting Matrices
Q_lqr = C' * C;          % Penalize position error heavily
R_lqr = 0.01;            % Cheap control effort

% 2. Calculate LQR Gain K
K_lqr = lqr(A, B, Q_lqr, R_lqr);
Nbar_lqr = -1 / (C * ((A - B*K_lqr) \ B));

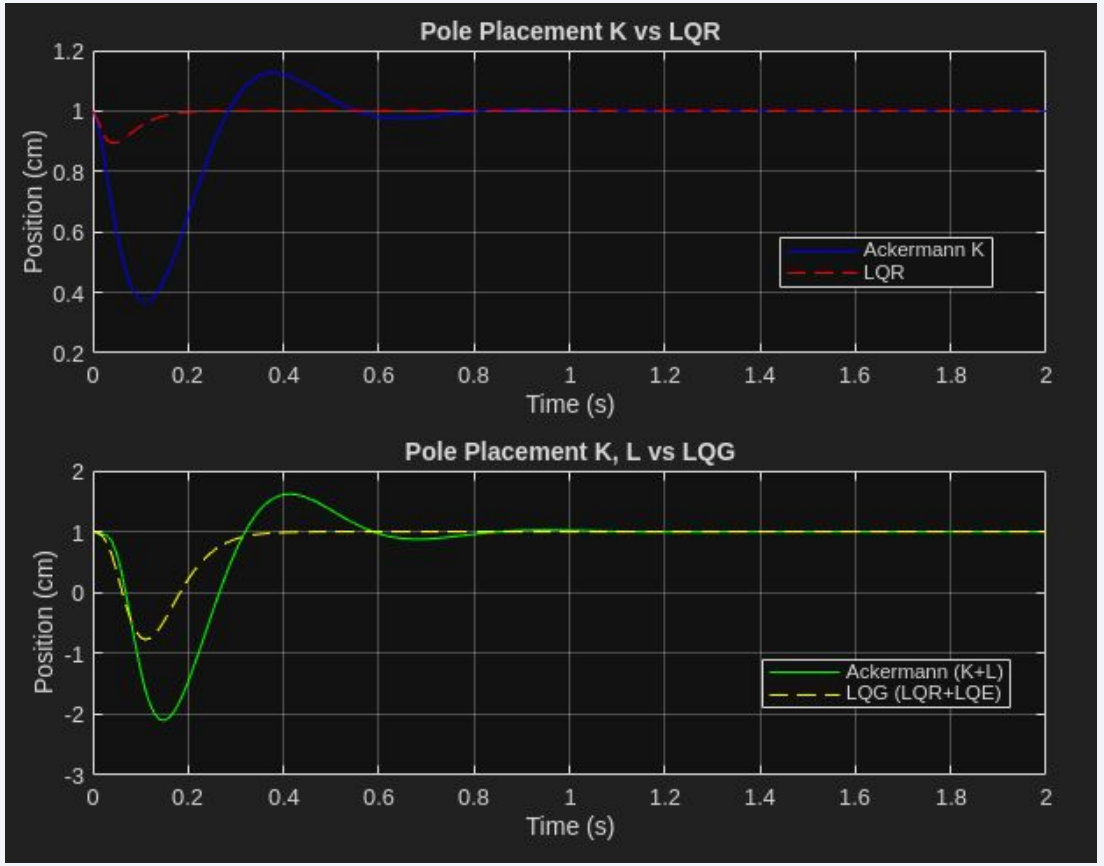
% 3. LQR Closed-Loop Simulation (Without Observer)
sys_lqr = ss(A - B*K_lqr, B*Nbar_lqr, C, 0);
[y_lqr, ~, x_lqr] = lsim(sys_lqr, r, t, x0);
```

```
%% [Part 5] LQE & LQG Design (Kalman Filter + LQR)
% 1. Noise Covariances
G_noise = B;             % Process noise enters at input
Qn = 0.01;               % Process noise variance
Rn = 0.1;                % Measurement noise variance

% 2. Calculate Kalman Gain L
[L_lqe, P_lqe, E_lqe] = lqe(A, G_noise, C, Qn, Rn);

% 3. LQG Augmented System Simulation (LQR + Kalman)
At_lqg = [A - B*K_lqr,      B*K_lqr;
          zeros(size(A)),   A - L_lqe*C];
Bt_lqg = [B * Nbar_lqr;
          zeros(size(B))];
Ct_lqg = [C, zeros(size(C))];

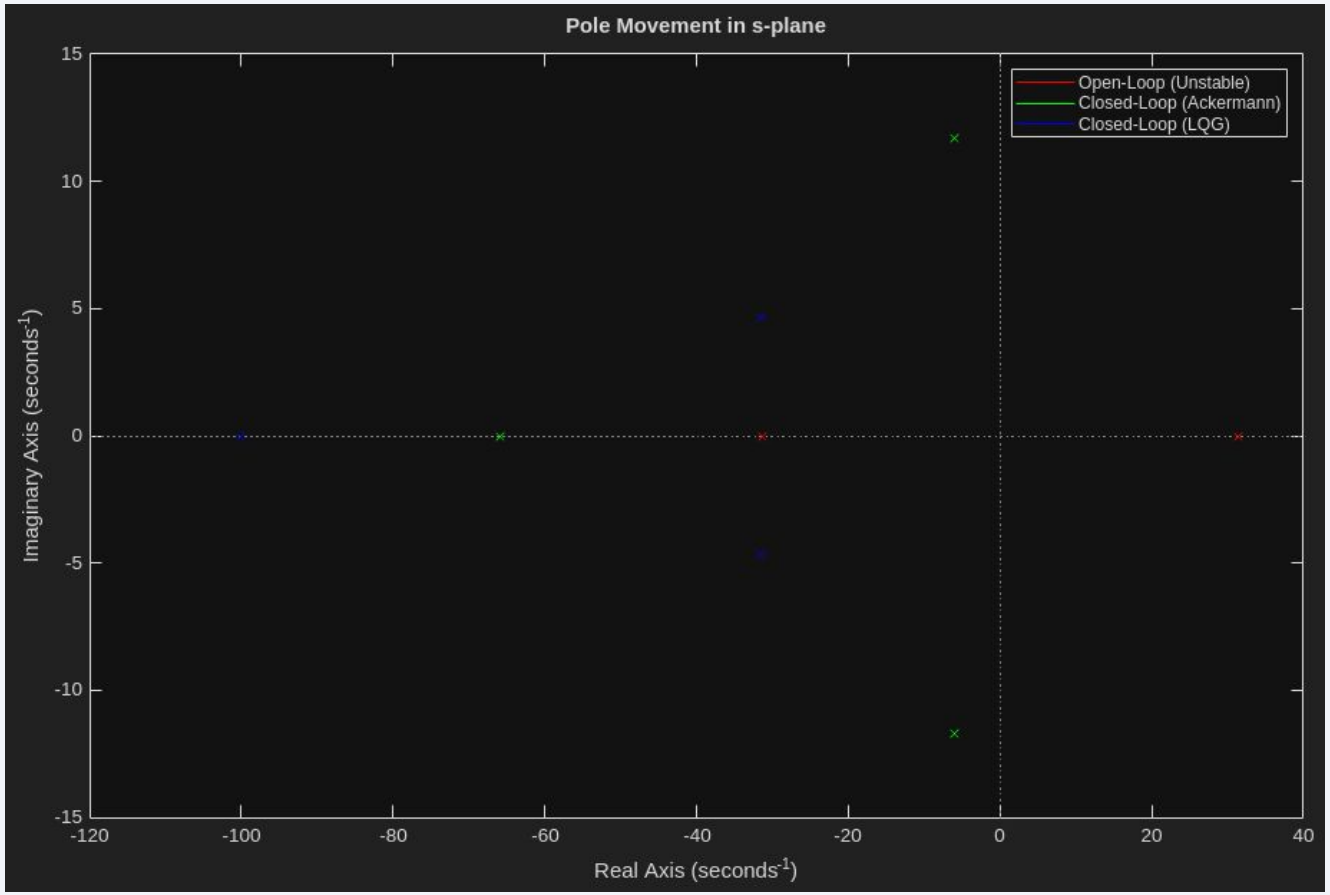
sys_lqg = ss(At_lqg, Bt_lqg, Ct_lqg, 0);
[y_lqg, ~, x_lqg] = lsim(sys_lqg, r, t, [x0; e]);
```



Ackermann은 원하는 위치로 극점을 강제로 이동시키므로
에너지가 크게 소모되어서 오버슈트가 크게 발생

-> 최적제어 기법인 LQG(LQR + LQE)은 에너지 소모와 오차 사이의 가중치를 계산하여 최적의 극점을 스스로 찾기 때문에 훨씬 부드럽고 안정적인 성능을 보여줌.

5. CT Simulation : 최적 제어




```
--- Open-Loop Poles ---
   31.3209
  -31.3209
 -100.0000

--- Closed-Loop Poles (Controller) ---
Ackermann:
 -65.7967 + 0.0000i
  -6.0000 +11.7119i
  -6.0000 -11.7119i

LQR:
   1.0e+02 *

  -1.0005 + 0.0000i
  -0.3159 + 0.0467i
  -0.3159 - 0.0467i

--- Error Dynamics Poles (Observer) ---
Ackermann:
   1.0e+02 *

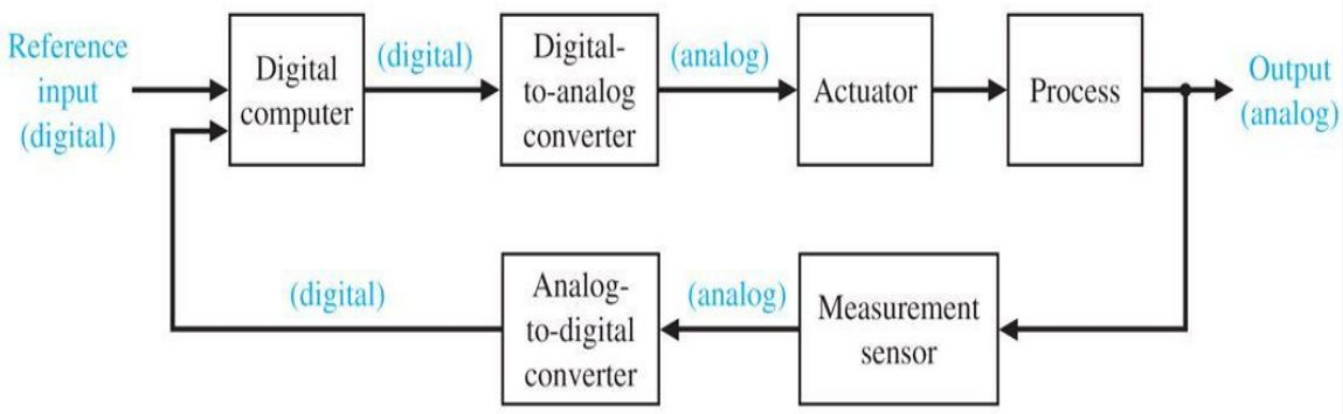
  -2.5374 + 0.0000i
  -0.3000 + 0.4093i
  -0.3000 - 0.4093i

Kalman:
   1.0e+02 *

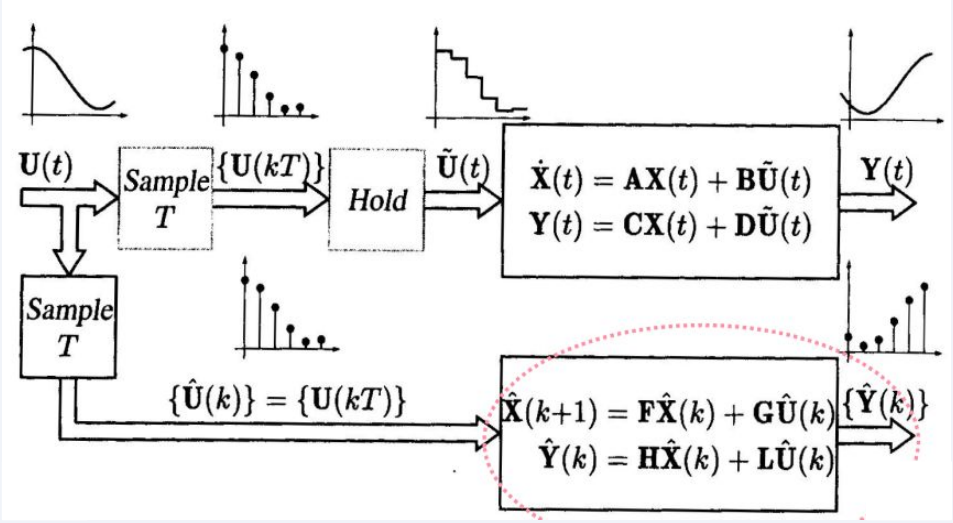
  -0.3132 + 0.0015i
  -0.3132 - 0.0015i
  -1.0000 + 0.0000i
```

LQR은 에너지를 최소한으로 쓰면서 안정화하므로, 이미 안정적인 개루프 극점은 놔두거나 근처에 둔다. 불안정 극점만 최소한의 힘을 들여 좌반면의 거울상 위치로 넘긴다.
Kalman filter(LQE) 또한 노이즈 증폭을 막기 위해 억지로 극점을 멀리 보내지 않고 필요한 만큼만 움직인다.
-> 본 프로젝트에서는 성능보다 제어시스템의 기본적인 개념 이해와 설계에 중점을 맞춰 Ackermann 방식을 채택

6. DT Simulation : 시스템 이산화



실제 시스템은 컴퓨터로 다루므로 이산 시간 모델로 변환해야한다



ADC(샘플링) , DAC(ZOH) 활용

$$x(t) = e^{At}x(0) + A^{-1}(e^{At} - I)bu$$

ZIR + ZSR

$$X[(k+1)T] = e^{AT}X[kT] + BU[kT]A^{-1}(e^{AT} - I)$$

G H

샘플링 주기에 따라 상태 방정식이 달라진다

6. DT Simulation : 시스템 이산화

```
%% [Part 2] 시스템 이산화 (ZOH)
dt = 0.02;                  % 샘플링 타임
t_dt = 0:dt:2;              % DT 전용 시간 벡터
n = length(t_dt);

% 행렬 지수 함수를 이용한 정확한 이산화
I = eye(3); % 3x3 단위 행렬

G_mat = expm(A * dt);
H_mat = A \ (G_mat - I) * B;
% sys_d = c2d(sys_open, dt, 'zoh');
% G = sys_d.A;
% H = sys_d.B;
```

$$\exp(TA) = e^{TA} = \mathrm{I} + \frac{T}{1!}\mathrm{A} + \frac{T^2}{2!}\mathrm{A}^2 + \frac{T^3}{3!}\mathrm{A}^3 + \frac{T^4}{4!}\mathrm{A}^4 + \cdots$$

상태 exp(A*dt)를 하면 행렬 A 원소에 e를 취하게 됨
expm(A*dt)로 해주어야 원하는 테일러 급수를 행렬 단위로 계산해준다

c2d 함수로 대체 가능

6. DT Simulation : 제어기/관측기 설계

kit 국립금오공과대학교 Kumoh National Institute of Technology

```
%% [Part 3] DT 전용 제어기/관측기 완벽 재설계 (Before the Loop!)
% 1. 제어기 극점 Z-매핑 및 K_dt 산출
p_z = exp([cp1 cp2 cp3] * dt);

% 가제어성 행렬 : [H, GH, G^2H]
C_c = [H_mat, G_mat*H_mat, (G_mat^2)*H_mat];

% 특성 방정식 행렬 다항식 Phi(G) 계산
Phi_G = (G_mat - p_z(1)*I) * (G_mat - p_z(2)*I) * (G_mat - p_z(3)*I);

% 공식 적용: K = [0 0 1] * C^(-1) * Phi(G)
K_dt = [0, 0, 1] * inv(C_c) * Phi_G;
% K_dt = place(G_mat, H_mat, p_z);  대체 가능


% 2. DT 전용 피드포워드 게인(Nbar_dt) 산출
% (DT의 정상 상태 오차 공식은 A 대신 (G - I)를 사용합니다)
Z_vec = [0; 0; 0; 1];
N_mat = inv([G_mat - eye(3), H_mat; C, 0]) * Z_vec;
Nbar_dt = N_mat(4) + K_dt * N_mat(1:3);


% 3. 관측기 극점 Z-매핑 및 L_dt 산출
op_z = exp([op1 op2 op3] * dt);

% 쌍대성 시스템 치환: G_dual = G^T, H_dual = C^T
G_dual = G_mat';
H_dual = C';

% 쌍대 시스템의 가제어성 행렬 (가관측성 행렬의 전치 형태)
C_o = [H_dual, G_dual*H_dual, (G_dual^2)*H_dual];

% 특성 방정식 행렬 다항식 Phi(G_dual) 계산
Phi_G_dual = (G_dual - op_z(1)*I) * (G_dual - op_z(2)*I) * (G_dual - op_z(3)*I);

% 공식 적용 후 전치(Transpose)하여 최종 L 획득
L_dt = ([0, 0, 1] * inv(C_o) * Phi_G_dual)';
% L_dt = place(G_mat', C', op_z)';  대체 가능
```

```
% Main Control Loop
for k = 1:n
    % 1. 데이터 저장 (미래로 가기 전, 현재 상태 '그대로' 저장!)
    y = C * x;                  % 현재 실제 위치
    y_hat = C * x_hat;          % 현재 추정 위치
    y_log(k) = y;
    y_hat_log(k) = y_hat;       % 추정 출력 전용 로그에 저장
    x_hat_log(k, :) = x_hat';   % (전류 등을 나중에 보기 위해 상태변수도 별도 저장)

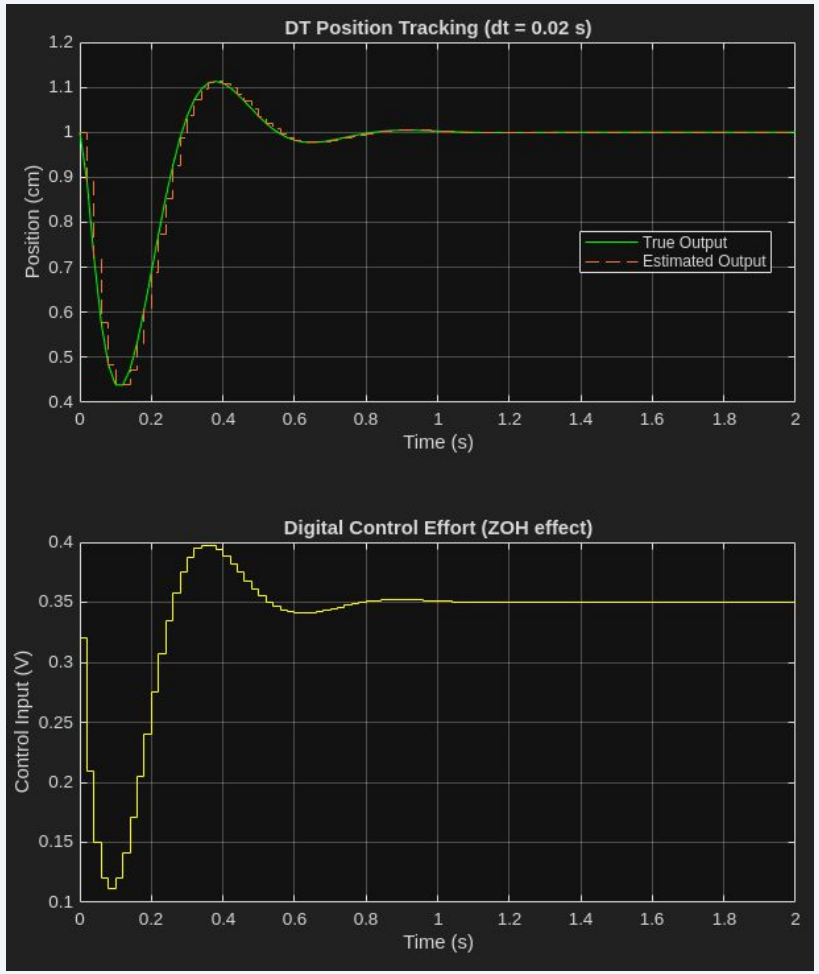
    % 2. 제어 법칙 및 측정
    u = -K_dt * x_hat + Nbar_dt * ref;
    u_log(k) = u;

    % 3. 상태 업데이트 (다음 스텝 k+1 로 이동)
    x = G_mat * x + H_mat * u;
    x_hat = G_mat * x_hat + H_mat * u + L_dt * (y - y_hat);
end
```

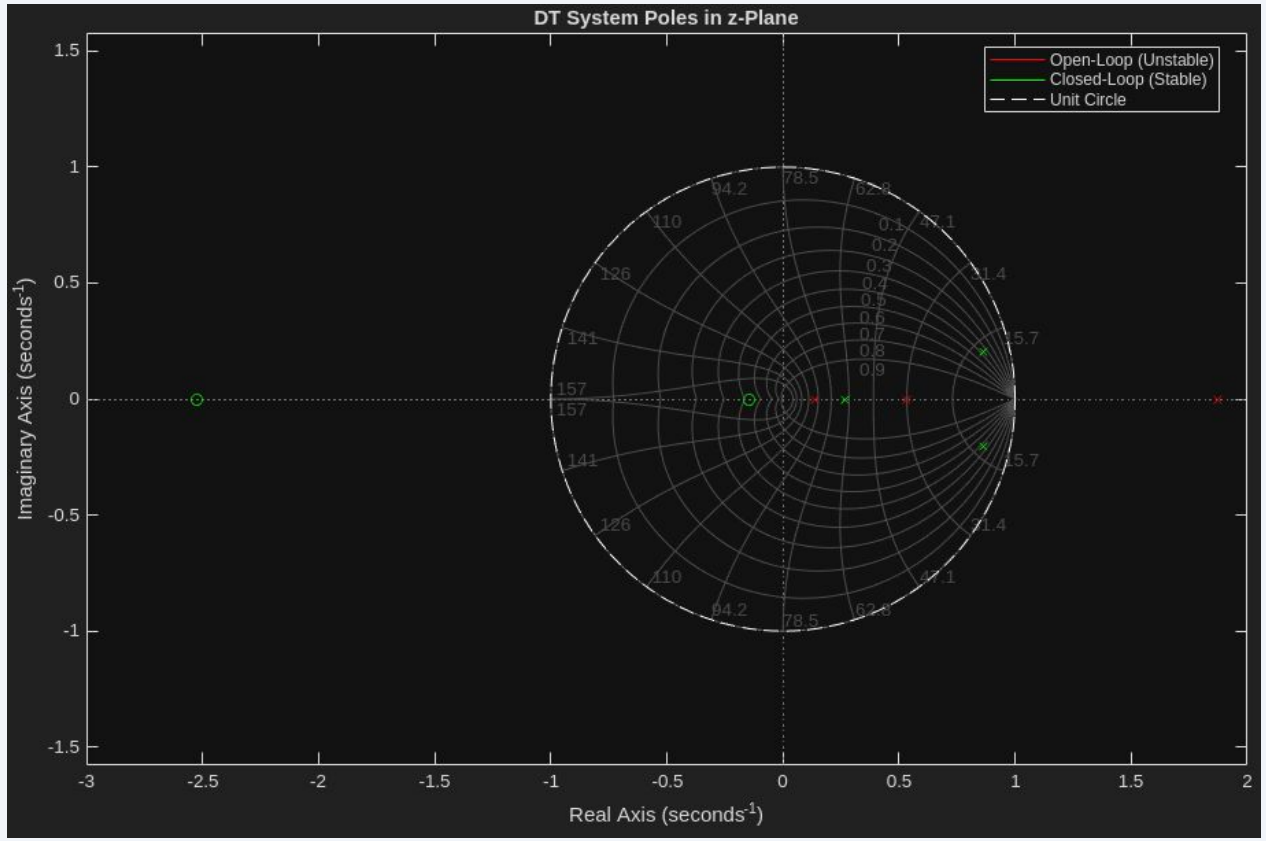
CT에서 채택한 Ackermann 방식 제어기 및 관측기 설계를 DT에서 반복

추후 임베디드 보드 포팅을 위해서 극점 배치와 상태 공간 시뮬레이션을 코드로 직접 구현 (acker 함수 -> Ackermann 공식, lsim 함수 -> for문)

6. DT Simulation : T = 0.02s



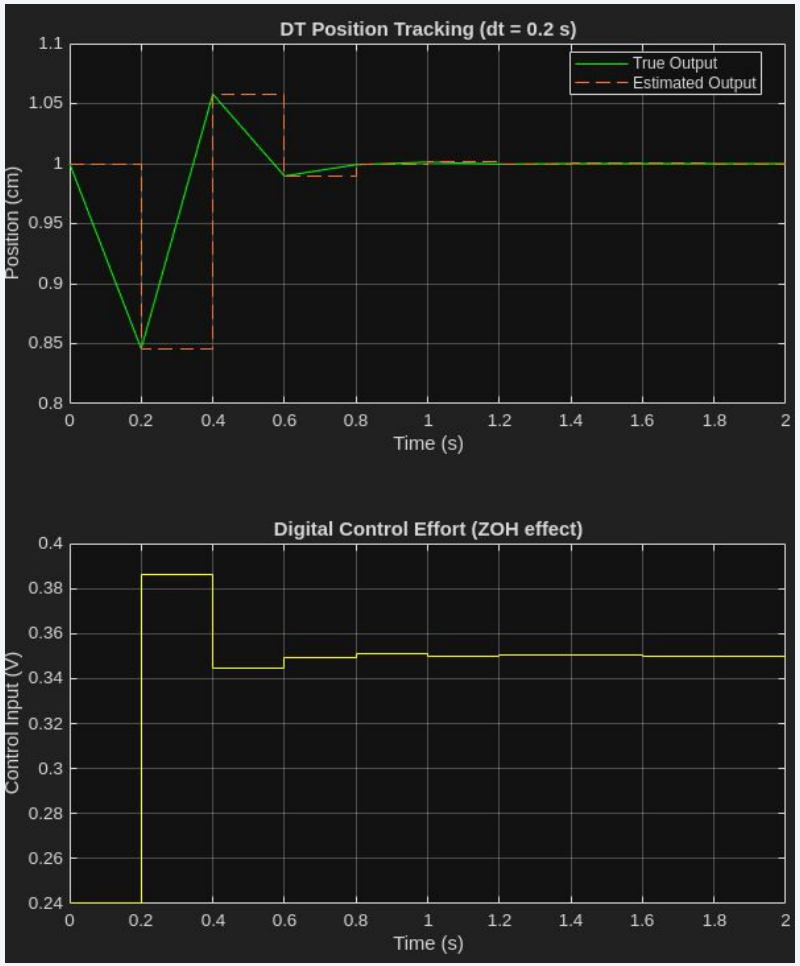
(ZOH를 시각화 하기 위해 plot -> stairs 함수로 그래프를 그림)



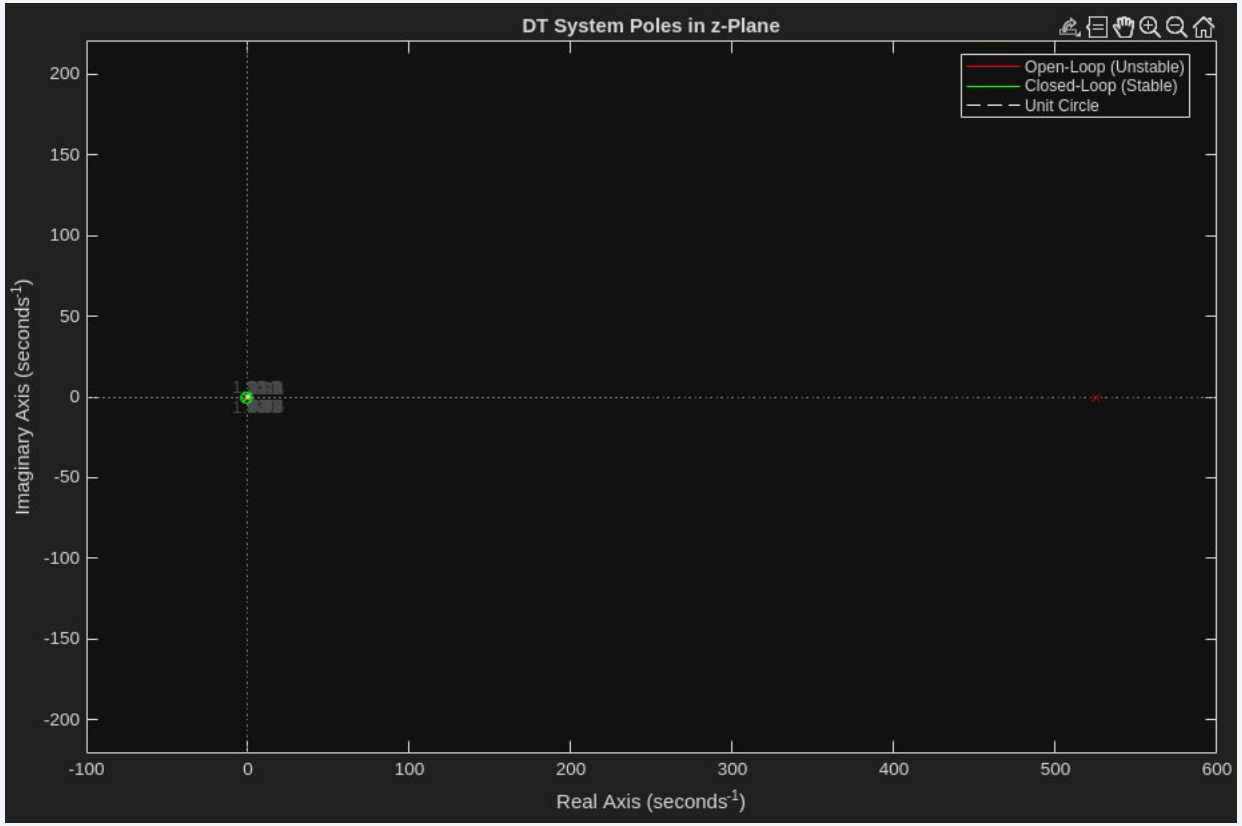
샘플링 주기 0.02초에서는 모든 극점이 단위원 내에 있으므로 안정.

관측기 초기 추정값을 실제 초기값과 똑같이 하여 초기 상태 오차가 0이지만, CT와 다르게 DT에서는 ZOH로 관측기가 실제값을 완벽하게 따라가지는 못한다.

6. DT Simulation : T = 0.2s



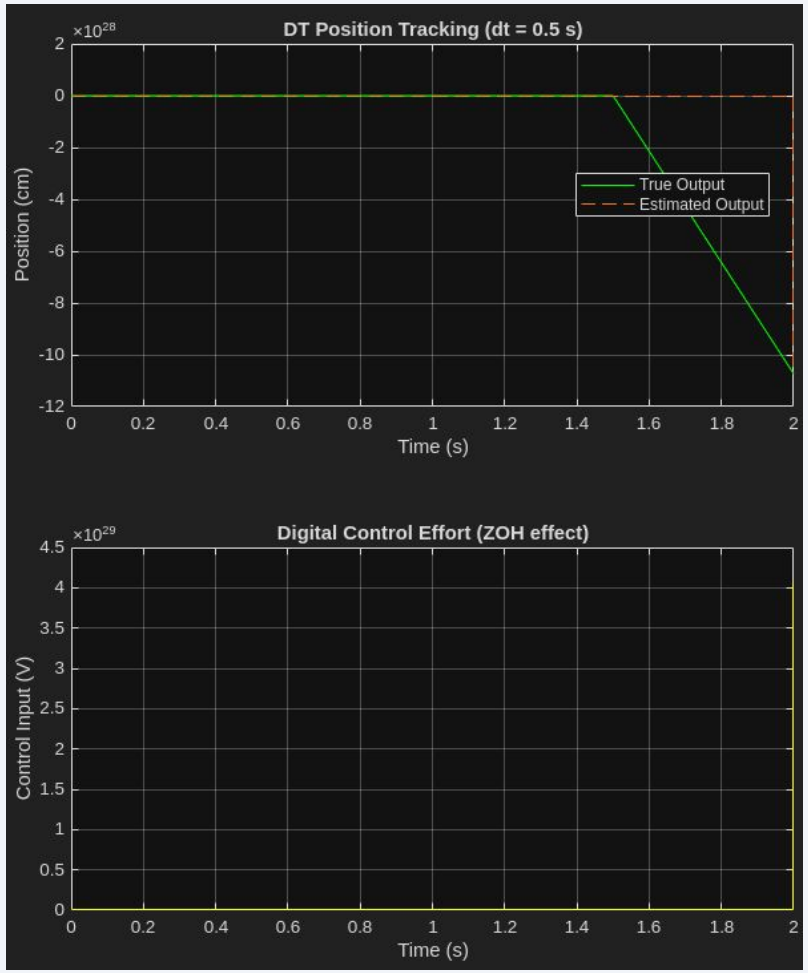
(ZOH를 시각화 하기 위해 plot -> stairs 함수로 그래프를 그림)



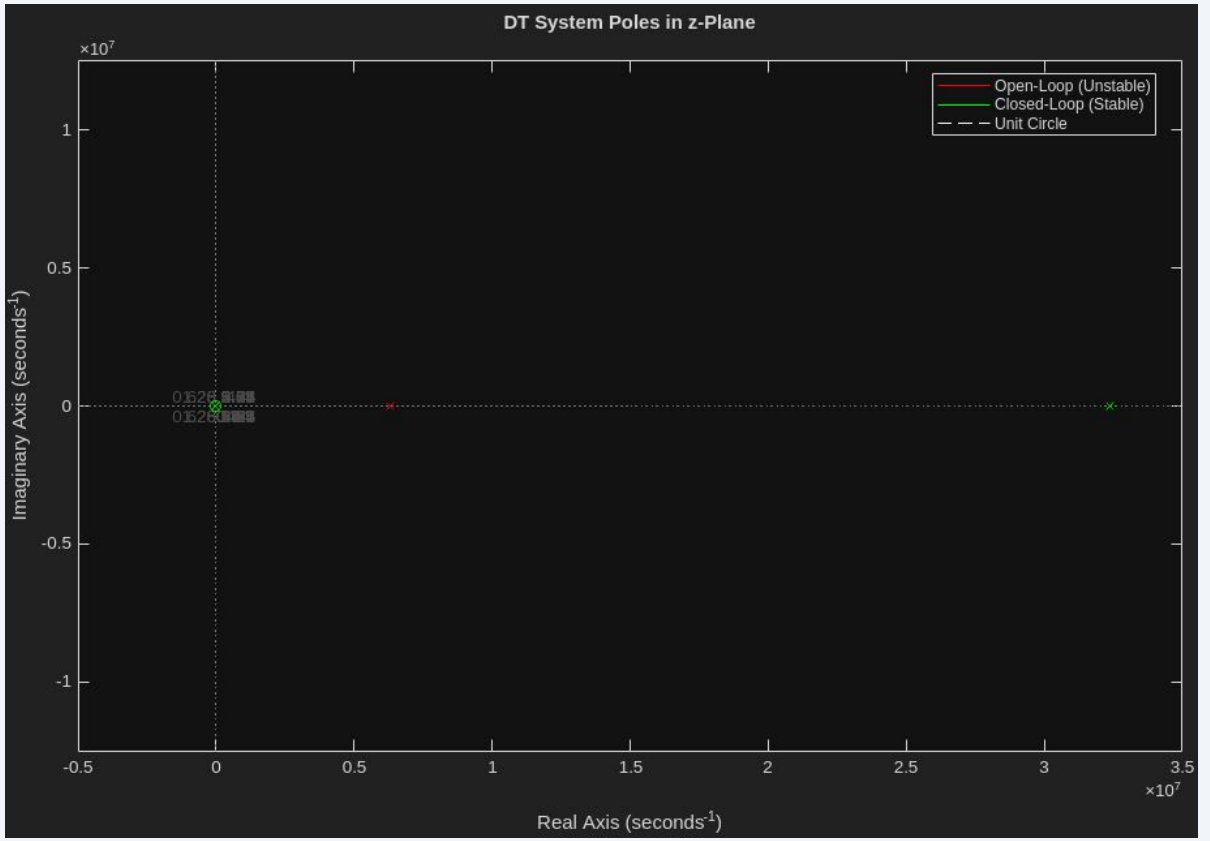
샘플링 주기 0.2초에서는 모든 극점이 단위원 내에 있으므로 안정?

수학적으로는 안정하나, 실제 임베디드 환경에서는 불안정.
True ouput도 연속적으로 그리면 다르게 나올 것이다

6. DT Simulation : T = 0.5s

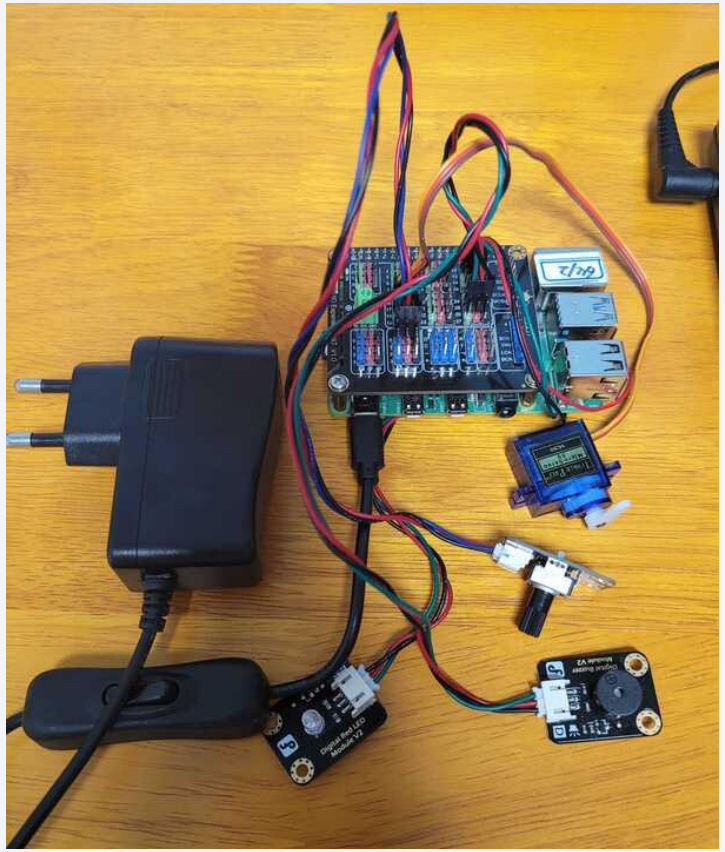


(ZOH를 시각화 하기 위해 plot -> stairs 함수로 그래프를 그림)



샘플링 주기 0.5초에서는 한 극점이 단위원 밖에 있으므로 불안정.

7. DT Realization : HIL 시스템 모델링



Hardware Components

가변저항: 목표치 r 입력 (4초에 한번씩)

서보모터: 출력 y 모사 (0~4cm 매핑)

라즈베리파이: 제어기 & 관측기 연산

부저/LED: 안전 마진 및 정착 확인

수학적 모델과 실제 하드웨어를 실시간으로 연결하여 제어 성능을 검증하는 HIL (Hardware-in-the-Loop) 시뮬레이션 시스템 구축.

7. DT Realization : 파이프라인

kit 국립금오공과대학교 Kumoh National Institute of Technology

파일 구성

generate_params.py (수학 모델링 및 튜닝): 물리 파라미터와 제어 사양 정의, 극점 배치 및 ZOH 이산화를 통해 상태 공간 행렬을 자동 계산하는 알고리즘.

matrix_params.h (브릿지 헤더): 파이썬 산출 설계값과 평형점 초기 상태를 C언어 시스템으로 넘겨주는(SSOT) 매개체. 실행 시마다 자동 갱신 및 생성.

Makefile (빌드 자동화): 파이썬 스크립트 실행부터 C 코드 컴파일까지 복잡한 종속성 트리를 한 번에 처리하여 빌드 효율을 극대화하는 설정 파일.

main.c (실시간 제어 본체): HIL 시뮬레이션 메인 루프. 센서 입력 필터링, 상태 피드백 연산 및 물리적 출력 제어를 수행하는 임베디드 애플리케이션.

시스템 실행 흐름 (Workflow)

Step 1: 제어기 사양 튜닝 - 샘플링 주기(dt), 목표 성능(ts_c, Mp_c) 또는 평형점(y0)을 파이썬 스크립트에서 수정.

↓

Step 2: 원클릭 빌드 - 터미널에서 make 실행하여 파이썬 연산, 헤더 갱신, C 컴파일 및 실행 파일 생성을 일괄 처리.

↓

Step 3: HIL 시뮬레이션 가동 - ./digital_twin 실행 후 가변저항을 조작하여 실시간 타겟 위치(Step Input) 인가.

↓

Step 4: 실시간 검증 및 지표 확인

- 제어 응답: 수학적 동역학 궤적 추종 확인
- 안전성: 발산 시 E-Stop 및 부저 알림 작동
- 성능 평가: 안착 시 LED 점등 및 ts 데이터 출력

7. DT Realization : MATLAB 코드 포팅

kit 국립금오공과대학교 Kumoh National Institute of Technology

MATLAB에서 그대로 넘어온 로직

상태 피드백 & 피드포워드

[MATLAB]

u = -K_dt * x_hat + Nbar_dt * ref;

[C Code]

u = -(K[0]*x_hat[0] + ...) + (Nbar * r_active);

행렬 연산을 C언어 스칼라 연산으로 전개. 수학적 구조는 100% 동일하게 유지되어 제어 성능의 연속성 보장.

상태 관측기 업데이트

[MATLAB]

x_hat = G*x_hat + H*u + L*(y-y_hat);

[C Code]

next_x_hat[i] = G[i][0]*x_hat[0]... + H[i]*u;

ZOH 이산화 모델 x(k+1) = Gx(k) + Hu(k) 기반의 상태 추정 알고리즘을 실시간 루프 내 그대로 구현.

Bumpless Transfer (범플리스)

[MATLAB]

x0_obs = x0; (초기 평형점 설정)

[C Code]

double x_hat[3] = {x10, x20, x30};

초기 구동 시 제어 충격 방지. 실제 물리 상태와 관측기 상태를 평형점으로 일치시키는 안전 로직 적용.

```
// 2. 물리 연산 및 추정
double y = x[0];
double y_hat = x_hat[0];

double u = -(K[0]*x_hat[0] + K[1]*x_hat[1] + K[2]*x_hat[2]) + (Nbar * r_active);
```

```
double next_x[3], next_x_hat[3];
for(int i=0; i<3; i++) {
    next_x[i] = G[i][0]*x[0] + G[i][1]*x[1] + G[i][2]*x[2] + H[i]*u;
    next_x_hat[i] = G[i][0]*x_hat[0] + G[i][1]*x_hat[1] + G[i][2]*x_hat[2]
                  + H[i]*u + L[i]*(y - y_hat);
}
for(int i=0; i<3; i++) {
    x[i] = next_x[i];
    x_hat[i] = next_x_hat[i];
}
```

7. DT Realization : MATLAB 코드 포팅

kit 국립금오공과대학교 Kumoh National Institute of Technology

라이브러리 선정: WiringPi → pigpio

1. 타이밍 정밀도 및 Jitter 억제:
1ms 주기 제어 시 OS 스케줄링 간섭은 치명적. **pigpio**는 하드웨어 타이머와 데몬을 활용해 칼같은 1ms 루프를 보장함 (WiringPi 대비 압도적 정밀도).

2. DMA 기반 고품질 PWM (서보모터 떨림 방지):
소프트웨어 PWM의 펄스 흔들림으로 인한 액추에이터 불안정성 제거. DMA(Direct Memory Access)를 통한 깨끗한 파형 생성으로 정밀 제어 구현.

3. 컴파일 및 환경 설정 특이사항:
병렬 처리를 위해 컴파일 시 **-pthread -lrt** 옵션 필수. 커널 보안 정책 충돌 방지를 위한 최신 버전 업그레이드 권장.

스케줄링 및 입력 방식의 전환

● **실시간 스케줄링:**
MATLAB: 사전에 정의된 고정 시간 배열 기반 finite 루프
C Code: **gpioTick()** 기반 경과 시간 측정 및 **gpioDelay**를 통한 정확한 샘플링(Ts) 보장 무한 루프

```
uint32_t elapsed = gpioTick() - start_time;
if (elapsed < loop_time_us) gpioDelay(loop_time_us - elapsed);
```

● **물리적 센서 연동 (I2C ADC):**
MATLAB: 수식으로 생성된 이상적 계단 함수 배열 사용
C Code: 가변저항 값을 **i2cReadWordData**로 읽어 물리 거리 매핑 및 4초 주기 소프트웨어 스텝 제너레이터 구현

```
// 1. 목표치 읽기 및 소프트웨어 스텝 생성기
raw = i2cReadWordData(i2c_h, A0_VAL_REG);
adc_val = ((raw & 0xFF) << 8) | ((raw >> 8) & 0xFF);
r_preview = (adc_val / 4095.0) * 0.04;

if (gpioTick() - last_step_time > 4000000) {
    double step_size = fabs(r_preview - r_active);
```

7. DT Realization : MATLAB 코드 포팅

kit 국립금오공과대학교 Kumoh National Institute of Technology

추상적 제어 출력의 물리 매핑

MATLAB: 연산된 상태 y를 배열에 저장(`y_log(k)=y;`)하는 것으로 종료
C Code: 가상 위치 y(0~4cm)를 서보모터 각도(0~180°)로 변환 후, PWM 펄스 폭 신호(`gpioServo`)로 매핑하여 실제 하드웨어 구동

```
// 3. 서보모터 출력
double angle = (y / 0.04) * 180.0;
if (angle > 180) angle = 180;
if (angle < 0) angle = 0;
gpioServo(SERVO_PIN, 500 + (int)(angle * 11.11));
```

안전 로직 및 실시간 성능 지표

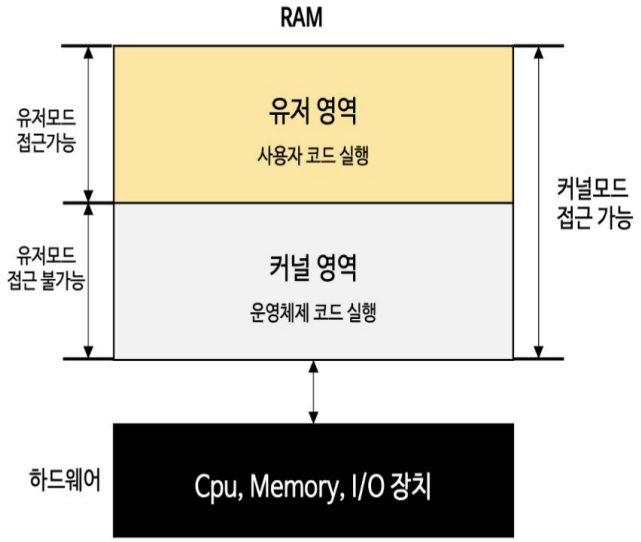
● **긴급 차단 시스템 (E-Stop):**
MATLAB: 불안정 시 단순 변수 발산 (물리적 피해 없음)
C Code: NaN 발생, 물리적 한계 초과, 과도한 입력(`|u|>50`) 시 즉각 부저 울림 및 제어 강제 종료 방어 로직

```
// [긴급 차단(E-Stop) 로직] 시스템 발산 감지
// ============================================================
// 위치가 물리적 한계(0cm ~ 4cm)를 아득히 벗어나거나 수학적 NaN에 빠진 경우
if (isnan(y) || y < 0 || y > 0.04 || fabs(u) > 50.0) {
    printf("\n\n\033[31m[긴급 경고] 시스템 발산 감지! (y=%.2fcm, u=%.1fV)\033[0m\n", y*100.0, u);
    printf("샘플링 주기가 너무 길거나 불안정합니다. 제어를 강제 차단합니다!\n");
    gpioWrite(BUZZER_PIN, 1); // 삐- 부저 울림
    gpioDelay(1000000); // 1초 대기
    break; // 제어 루프를 즉시 탈출
}
```

● **On-the-Fly 성능 지표 추출:**
MATLAB: 시뮬레이션 종료 후 `stepinfo`를 통한 사후 분석
C Code: 실시간 2% 오차 범위 판별, 50틱 연속 유지 시 실제 정착 시간(ts) 즉시 계산 및 LED/터미널 피드백

```
// 4. 정착 시간 판독
if (fabs(r_active - y) <= allowed_error) {
    settle_counter++;
    if (settle_counter > 50 && !is_settled) {
        is_settled = 1;
        double settle_time_s = ((gpioTick() - step_start_time) / 1000000.0) - 50 * TS;
```

7. DT Realization : 커널 모듈 작성 (ver.2)



Hard-Real time을 위해 커널 모듈 작성

연산 주체의 완벽한 분리

무거운 연산은 커널이 하지 않는다

- 고정소수점은 FPU를 못 쓰는 저사양 환경의 고육지책
- 현 구조는 무거운 연산을 유저 스페이스(main.c)로 분리하여 유저 영역에서 FPU로 double 연산 실시간 처리

타이머 용도에는 최적화가 필요 없다

- 커널 모듈(mbs_timer.c)은 제어 연산을 수행하지 않고, 단지 정수형 시간(TS_NS)을 세고 유저를 깨우는 순수 타이머 역할
- 수학적 연산 자체가 없으므로 최적화 대상도 존재하지 않음

정수 스케일링

나노초(ns) 단위 직결 설계

- 0.002초를 2,000,000L(정수)로 변환하여 매크로 전달
- 리눅스 HRTimer는 애초에 **나노초 단위 정수**를 기본값으로 요구하므로, 이가 시스템 표준에 가장 부합하며 복잡한 변환 오버헤드 제거

```
#define TS_NS 2000000L  // 커널 전용 타이머
```

Q-15 고정소수점 연산 최적화 대체

7. DT Realization : 커널 ISR 구현 (ver.2)

kit 국립금오공과대학교 Kumoh National Institute of Technology

1. 타이머 주도권: Software Delay → HRTimer

● 유저 모드: while 루프 내 `gpioDelay()` 기반. 스케줄러 간섭 시 치명적 지터(10~20ms) 발생

● **커널 하이브리드: 고해상도 타이머(HRTimer)** 사용. 하드웨어 인터럽트 레벨에서 정확히 2ms(T) 주기 강제 실행, Hard Real-Time 보장

```
// 1. HRTimer 콜백 (정확히 TS_NS 마다 실행)
// ============================================================
static enum hrtimer_restart timer_callback(struct hrtimer *timer) {
    ktime_t now = ktime_get();

    // 플래그를 세우고 대기 중인 유저 프로그램(main.c)을 깨움
    sync_flag = 1;
    wake_up(&sync_wq);

    hrtimer_forward(timer, now, kt_interval);
    return HRTIMER_RESTART;
}
```

2. 프로세스 동기화: Wait Queue 구조

● **커널 스페이스:** `wait_queue` 생성. 2ms마다 `wake_up()` 호출하여 유저 프로그램 기상

● **유저 스페이스:** `read()` 시스템 콜 호출 후 Blocking(수면). CPU 점유율 0% 유지하다 인터럽트 즉시 제어 연산 수행

```
// 무한 루프
while (keep_running) {
    // [핵심] 커널 타이머가 깨워줄 때까지 여기서 수면 (Blocking)
    if (read(fd, sync_buf, 1) < 0) break;
```

3. 커널-유저 브릿지: 가상 문자 디바이스

● **가상 디바이스 (/dev/mbs_sync):** "모든 것은 파일이다" 철학 구현. 표준 POSIX 인터페이스 (`open/read`)로 커널의 정밀 타이머 자원 활용

```
int fd = open("/dev/mbs_sync", O_RDONLY);
if (fd < 0) {
    printf("커널 모듈 장치 열기 실패. 'sudo insmod
    gpioTerminate();
    return -1;
}
char sync_buf[1];
```

4. 단일 진실 공급원(SSOT): 매크로 분리

● **매크로 전처리기:** 파이썬에서 헤더 생성 시 `#ifndef __KERNEL__` 사용. 커널은 정수 타이머, 유저(main.c)는 부동소수점 매크로만 선택적으로 읽어 동기화

```
#define TS_NS 2000000L  // 커널 전용 타이머

// ============================================================
#ifndef __KERNEL__
// [유저 전용] 커널은 이 아래의 부동소수점(double)을 무시함

const double TS = 0.002;
```

💡 철저한 역할 분담 (Separation of Concerns)

- **커널 스페이스 (Tick 생성):** 시간에 민감한 하드웨어 인터럽트 및 정밀 맥박 생성 전담
- **유저 스페이스 (수학적 두뇌):** 복잡한 행렬 연산, I2C/PWM 하드웨어 제어 로직 100% 유지 (커널 패닉 방지)

7. DT Realization : RTOS 구현 (ver.3)

kit 국립금오공과대학교 Kumoh National Institute of Technology

1. 스케줄링 정책 격상 (FIFO)

● **독점형 스케줄링:** `SCHED_FIFO` 정책 적용

● 기본 시분할 방식에서 벗어나 제어 프로그램이 CPU를 자발적으로 내려놓기 전까지 OS의 강제 선점 차단

타 프로세스 간섭에 의한 타이머 지터(Jitter) 최소화

2. 우선순위 극대화 (P=99)

● **최고 우선순위:** 시스템 가용 최상위 수치(**99**) 부여

● 시스템 핵심 프로세스를 제외한 모든 유저 프로그램 중 무조건적인 1순위 실행 권한 획득

리소스를 가장 먼저 선점하는 "VIP 프로세스" 자격 확보 (sudo 권한 필수)

3. 메모리 락 (mlockall)

● **물리 RAM 상주:** Page Fault에 의한 디스크 스왑 원천 차단

● 제어 프로그램이 사용하는 모든 메모리를 물리 램에 고정시켜 접근 지연 시간 제거

비정기적으로 발생하는 수십~수백 ms의 메모리 로딩 지연 원천 방지

```c
#include <sched.h>      // [RTOS 추가] 실시간 스케줄링 헤더
#include <sys/mman.h>   // [RTOS 추가] 메모리 락 헤더
```

```c
void set_realtime_priority() {
    // 1. 메모리 락 (Memory Lock)
    // OS가 메모리를 디스크로 스왑(Swap)하면서 발생하는 치명적인 지연을 원천 차단
    if (mlockall(MCL_CURRENT | MCL_FUTURE) == -1) {
        printf("경고: mlockall 실패 (메모리 락 미적용)\n");
    }

    // 2. 실시간 스케줄러(SCHED_FIFO) 설정 및 우선순위 99(최대) 부여
    // 일반 리눅스 프로세스(유튜브, 웹브라우저 등)가 CPU를 뺏어가지 못하게 함
    struct sched_param sp;
    sp.sched_priority = 99;
```

💡 소프트웨어적 실시간성 (Firm Real-Time) 극대화

- **유저 RTOS 모드 (main2.c):** OS 스케줄러를 통제하여 실행 순위를 독점하나, 시간 측정은 여전히 유저 레벨 타이머에 의존
- **커널 하이브리드 (mbs_timer.c):** 시간(알람시계) 자체를 커널의 심장에 박아 넣은 구조로, Hard Real-Time을 달성하는 최상위 아키텍처

커널 코드 수정 없이 OS 스케줄러 최적화만으로 실시간성을 확보하는 실무적 접근

7. DT Realization : ver 3개 간 성능 비교

kit 국립금오공과대학교 Kumoh National Institute of Technology

```
demo@onetwo:~/DSS/Mechanical-ball_suspension$ sudo ./digital_twin

[MBS HIL System | T = 0.0020s | Step Period = 4.0s]
가변저항을 돌려 '예약'해두면, 4초마다 스텝 입력이 가해집니다.
(안전 기능 활성화됨: Ctrl+C로 안전 종료 지원)

(다음목표:1.00cm) 현재목표:1.00cm | 위치:1.06cm | 속도^: -0.01 | 전류^: 0.37 | 제어:  0.4V
>>> [정착 완료] 목표: 1.00cm | 2% 정착 시간: 0.513 s <<<
```

유저 모드 (main.c)

```
demo@onetwo:~/DSS/Mechanical-ball_suspension$ sudo ./digital_twin2
[SYSTEM] 하드 리얼타임(SCHED_FIFO, Priority=99) 모드 활성화 완료!

[MBS HIL System | T = 0.0020s | Step Period = 4.0s]
가변저항을 돌려 '예약'해두면, 4초마다 스텝 입력이 가해집니다.
(안전 기능 활성화됨: Ctrl+C로 안전 종료 지원)

(다음목표:1.00cm) 현재목표:1.00cm | 위치:1.05cm | 속도^: -0.01 | 전류^: 0.37 | 제어:  0.4V
>>> [정착 완료] 목표: 1.00cm | 2% 정착 시간: 0.499 s <<<
```

유저 RTOS 모드 (main2.c)

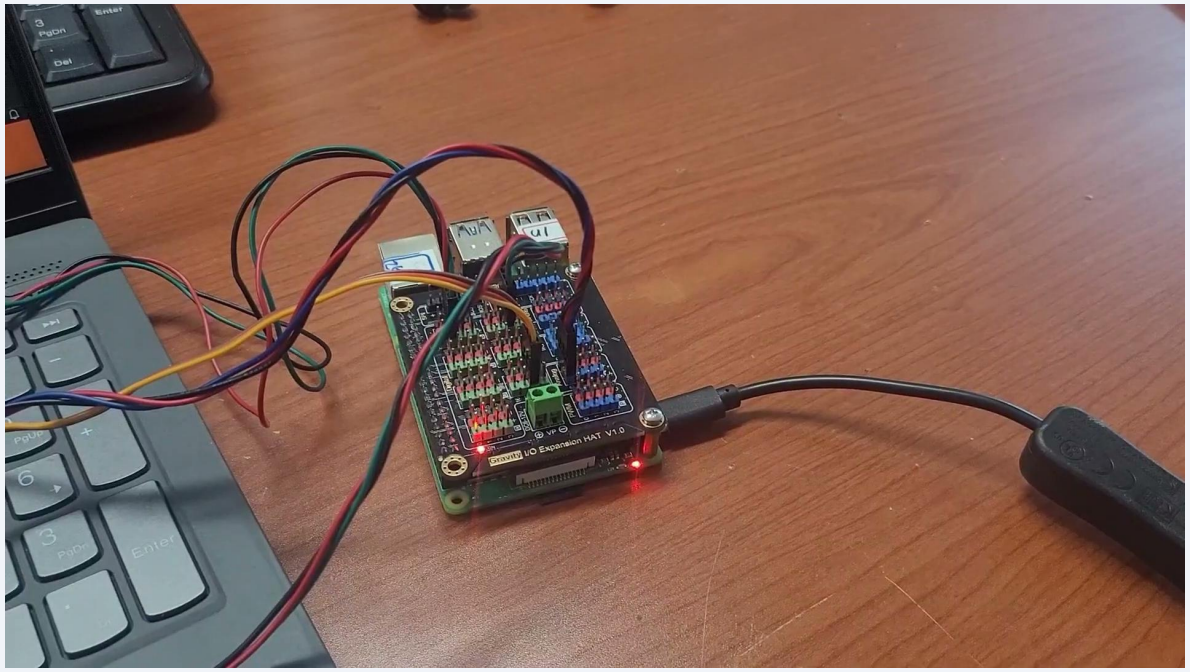
```
demo@onetwo:~/DSS/Mechanical-ball_suspension/kernel_ws$ sudo ./digital_twin

[MBS HIL System | T = 0.0020s | Step Period = 4.0s]
가변저항을 돌려 '예약'해두면, 4초마다 스텝 입력이 가해집니다.
(커널 하이브리드 모드 활성화됨 / Ctrl+C로 안전 종료 지원)

(다음목표:1.00cm) 현재목표:0.99cm | 위치:1.02cm | 속도^: -0.01 | 전류^: 0.36 | 제어:  0.4V
>>> [정착 완료] 목표: 0.99cm | 2% 정착 시간: 0.484 s <<<
```

커널 하이브리드 (main.c + mbs_timer.c)

7. DT Realization : 시연 영상



결론

커널 모드로 돌렸을 때 가장 성능이 우수,
평형점에서 크기가 큰 스텝 입력 인가 시 오버슈트 주의
(실제 시스템에서 물리적 허용 범위 초과 시 붕괴 위험)

8. 트러블슈팅

kit 국립금오공과대학교 Kumoh National Institute of Technology

파라미터 업데이트 효율성 개선: Python 스크립트 기반 행렬 자동 연산 및 Header 파일(.h) 자동 생성 시스템 구축 (Make 명령어 한 줄로 컴파일 완료).

가변저항 연속성에 따른 정착 시간 오류: 실제 손 움직임을 '램프 입력'으로 인식하여 타이머가 꼬이는 문제 발생. 입력값 변화 정지 시점을 '스텝 시작점'으로 간주하는 필터 알고리즘 도입.

부저 작동 불능 현상: 하드웨어 핀 접촉 불량 확인 및 핀 재설정을 통한 안정적 출력 확보.

9. 한계 및 후속연구

동작점 선형화의 한계

비선형성 : 직선으로 곡선을 대변하므로 평형점 미소영역 이탈 시 오차 증가. Matlab에서와 다르게 실제 모터는 발산 가능.

대응책 : 강인 제어(Robust), 게인 스케줄링, 혹은 피드백 선형화 기법 필요.

실시간 제어에서 데이터 저장

I/O 병목 : 실시간 루프 내 csv 파일 저장은 지터(Jitter)의 주범.

대응책 : pthread를 활용한 멀티스레딩(제어/로깅 분리), 커널 모드-유저 모드 간 링 버퍼(kfifo) 통신 고도화.

최적 제어를 통한 성능 향상

에너지 낭비 : Ackermann 공식을 활용한 극점 배치 방식은 무리한 제어 입력으로 오버슈트가 크게 발생.

대응책 : 최적 제어 기법인 LQG를 구현하여 에너지 최적화 및 노이즈 증폭 방지.

10. 결론

kit 국립금오공과대학교
Kumoh National Institute of Technology

제어 시스템 통찰

정밀 제어 공학의 실무적 통찰 획득

설계 역량

디지털 환경 이식 및 임베디드 설계 역량 확보

불안정한 물리 시스템을 수학적으로 선형화하고 안정화하는 전 과정을 직접 경험함.

향후 고도화된 실무 임베디드 제어 시스템 설계에 필수적인 엔지니어링 기반 마련.

참고 문헌

양연모, "CH_0_Discrete_SSstartup", (국립금오공과대학교 , 2026).
양연모, "M1_CH_11", (국립금오공과대학교 , 2026).
양연모, "M2_CH_13", (국립금오공과대학교 , 2026).